claude-windows / 6a82308f-69d6-43a8-9ea4-d8eaf497253d

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\6a82308f-69d6-43a8-9ea4-d8eaf497253d\subagents\workflows\wf_736be611-fb5\agent-a242ce8e220cc08f9.jsonl`
- SHA-256: `3767e0ea02b5940db21d2d104d239be501d99396ee69cb749b8b6f8c953c43de`
- Source modified: `2026-06-16T15:16:31+00:00`
- Imported at: `2026-07-05T16:48:27+00:00`
- project: `wf_736be611-fb5`
- session_id: `6a82308f-69d6-43a8-9ea4-d8eaf497253d`

Transcript

1. user (2026-06-16T15:14:30.292Z)

```
Adversarially VERIFY this finding from the real-footage rally-detection validation. Try hard to
REFUTE or show it is overstated, using ONLY the data (you MAY Read
output/local_vs_gemini_6.json). Default to holds=false if the numbers do not clearly support it.

CLAIM (windowing-TAL): mahadevpura-1 is an irreducible blob: W1's split-at-largest-inactivity-
gap logic cannot cut it because the clip is continuously active, so both the cap and W2 are no-
ops there.
EVIDENCE CITED: mahadevpura-1: baseline n=1 (mean 174.3s), W1 n=2 (mean 86.9s, max 101.9s -- far
above the 12s cap), W1+W2 unchanged n=2; matched=0, mean_iou=0.0 across all variants, 9 gem
rallies trapped in merges. W1's per-video mean window of 86.9s vs the 12s cap proves the cap
never fired -- no internal inactivity gap exists to split on.


REAL-FOOTAGE VALIDATION DATA (6 Kushagra/Yash June-12 clips, 1080p proxies, fully-LOCAL no-AI
WASB
windowing vs Gemini. Gemini = strong REFERENCE, NOT ground truth ? these 6 have no golden labels
EXCEPT mahadevpura-2 which the owner hand-labeled, see GROUND-TRUTH below).

Rally COUNT (Gemini | local baseline | local W1 cap=12 | local W1+W2 mc=1):
  GX010128: 39 | 15 | 78 | 75
  GX010129: 27 | 2  | 50 | 47
  GX020125: 11 | 6  | 18 | 15
  mahadevpura-0: 40 | 19 | 62 | 43
  mahadevpura-1: 9 | 1 | 2 | 2
  mahadevpura-2: 39 | 5 | 40 | 40
  AGGREGATE: Gemini 165 | baseline 48 (mean window 62.9s, 27 merge-groups, mIoU 0.23) |
             W1 250 (mean 10.8s, 10 merge-groups, mIoU 0.39) |
             W1+W2 222 (mean 11.5s, 10 merge-groups, mIoU 0.39, 6 gemini-only "misses")
  Per-video mean window (W1): GX010128 7.8s, GX010129 9.3s, GX020125 14.1s, mahadevpura-0 10.4s,
             mahadevpura-1 86.9s (the continuously-active blob W1 cannot cap ? no inactivity
gap),
             mahadevpura-2 13.8s. Gemini means ~5.7-7.2s.

GROUND-TRUTH (mahadevpura-2, 40 owner-labeled rallies, mean 5.7s, verified frame-aligned):
  F1@0.5 / F1@0.25 / mean-IoU:  Gemini 0.557/0.835/0.55 ; local W1+W2 0.150/0.500/0.31 ;
  local W1+W2+hard-cap 0.296/0.722/0.50 (best local). baseline 0.000/0.000/0.03.
  BOUNDARY ERROR (matched rallies, seconds): Gemini |?start| med 1.46 |?end| med 1.19 ;
  local-hardcap |?start| med 1.74 (~= Gemini!) |?end| med 3.12 (>> Gemini).
  => local's rally-START detection is ~at par with Gemini; the gap is the rally-END.
  IoU<->time: for a 5.7s rally, tIoU 0.5 tolerates only ~+-1.9s shift (the ~1-1.5s service
window
  nearly exhausts it -> tIoU 0.5 over-penalizes; owner wants a +-2s tolerance metric).

PRIOR GOLDEN DELTAS (n=6, then n=7 incl. mahadevpura-2): baseline F1 0.300 -> W1 0.439 (+0.139,
```

6/0)
  -> W1+W2 0.457 (+0.019, 6/0); hard-cap golden-neutral on n=6 (+0.006 n.s.) but became BEST on
n=7
  (continuously-active clips). All flags default-OFF, promoted only on measured LOVO lift +
sign-test.

WINDOWING KNOBS: a frame is "active" iff shuttle visible AND per-second-normalized velocity >
  velocity_thresh(0.02); active frames within merge_gap(2.0s) coalesce; min_window(2.0s). W1 =
  max_window_duration cap(12s) split at the largest internal inactivity gap. W1.5 hard-cap =
  midpoint-chop when no gap. W2 = net-crossings gate (drop windows with <1 net crossing).

2. assistant (2026-06-16T15:14:32.869Z)

I'll verify this claim by examining the actual data file.

3. user (2026-06-16T15:14:33.801Z)

File does not exist. Note: your current working directory is C:\Users\avidu\Projects\badminton-
highlight-indexer\.claude\worktrees\w2-safeguard.

4. assistant (2026-06-16T15:14:36.132Z)

The cited file does not exist at the expected path. Let me locate it.

5. user (2026-06-16T15:14:36.669Z)

No files found

6. user (2026-06-16T15:14:37.067Z)

No files found

7. user (2026-06-16T15:14:40.454Z)

backend\eval\gemini_refine.py
backend\pipeline\segmenters\gemini.py
backend\providers\gemini.py
eval_baselines\gemini_wrapper_2026-06-10.json
scratch\gemini_video_debugger.py
tests\test_gemini.py
tests\test_gemini_refine.py
tests\test_provider_gemini.py
tests__pycache__\test_gemini.cpython-313-pytest-9.0.3.pyc
tests__pycache__\test_gemini_refine.cpython-313-pytest-9.0.3.pyc
tests__pycache__\test_provider_gemini.cpython-313-pytest-9.0.3.pyc
backend\eval__pycache__\gemini_refine.cpython-313.pyc
backend\pipeline\segmenters__pycache__\gemini.cpython-313.pyc
backend\providers__pycache__\gemini.cpython-313.pyc

8. user (2026-06-16T15:14:40.464Z)

output\wasb\GX010128__db84f3e65318_wasb_wasbcache\det_raw.jsonl
output\wasb\GX010128__db84f3e65318_wasb_wasbcache\manifest.json
output\wasb\GX010128__db84f3e65318_wasb.csv
output\val_reels\GX010128 - Highlights.mp4
output\highlights_work\GX010128 - run.log
output\val_reels\GX010128 - run.log
output\wasb\GX010129__99a3e73afde3_wasb_wasbcache\det_raw.jsonl
output\wasb\GX010129__99a3e73afde3_wasb_wasbcache\manifest.json
output\wasb\GX010129__99a3e73afde3_wasb.csv
output\val_reels\GX010129 - Highlights.mp4
output\highlights_work\GX010129 - run.log
output\val_reels\GX010129 - run.log
output\wasb\mahadevpura-0__6460c2fc7647_wasb_wasbcache\det_raw.jsonl

```
output\wasb\mahadevpura-0__6460c2fc7647_wasb_wasbcache\manifest.json
output\wasb\mahadevpura-0__6460c2fc7647_wasb.csv
output\val.db
output\val_reels\mahadevpura-0 - Highlights.mp4
output\highlights_work\mahadevpura-0 - run.log
output\highlights_work\_batch_run.log
output\val_reels\mahadevpura-0 - run.log
output\val_reels\_batch_run.log
output\val_extract.log
```

9. user (2026-06-16T15:14:40.857Z)

```
output\wasb\mahadevpura-0__6460c2fc7647_wasb.csv
output\val_reels\mahadevpura-0 - Highlights.mp4
output\highlights_work\mahadevpura-0 - run.log
output\val_reels\mahadevpura-0 - run.log
```

10. assistant (2026-06-16T15:14:44.034Z)

The cited `output/local_vs_gemini_6.json` file does not exist in this worktree. The output directory contains WASB caches and reels but no comparison JSON. Let me check the other worktrees and the main repo, since this is a shared checkout.

11. user (2026-06-16T15:14:46.761Z)

```
No files found
```

12. user (2026-06-16T15:14:46.967Z)

```
/c/Users/avidu/Projects/badminton-highlight-indexer/output/local_vs_gemini_6.json
/c/Users/avidu/Projects/badminton-highlight-indexer/output/local_vs_gemini_6.md
/c/Users/avidu/Projects/badminton-highlight-indexer/output/local_vs_gemini_report.md
---WORKTREES---
agent-a771ff1d85e18f2ed
bhi-execpolicy
docs-rally-quality
gf-audit
gf-crossos
gf-p1
golden-fixtures
hardchop
local-noai
tier0-eval
tier1-windowing
w2-netcross
w2-safeguard
```

13. assistant (2026-06-16T15:14:49.422Z)

The file exists in the main repo checkout (not this worktree). Let me read it.

14. user (2026-06-16T15:14:50.026Z)


```
1       [
2         {
3           "name": "GX010128",
4           "vid": "db84f3e65318",
5           "csv": "C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\.claude\\worktrees\\w2-safeguard\\output\\wasb\\GX010128__db84f3e65318_wasb.csv",
6           "n_gem": 39,
7           "play_gem": 228.0,
8           "mean_gem": 5.846153846153846,
9           "variants": {
10            "baseline": {
```

```
            "n": 15,
            "play": 711.4614614614616,
            "mean_dur": 47.430764097430774,
            "max_dur": 133.2499165832499,
            "count_ratio": 0.38461538461538464,
            "matched": 4,
            "local_only": 1,
            "gemini_only": 0,
            "splits": 0,
            "merges": 10,
            "mean_iou": 0.3564105364413525,
            "gem_rallies_in_merges": 35
          },
          "W1": {
            "n": 78,
            "play": 605.1051051051056,
            "mean_dur": 7.757757757757765,
            "max_dur": 16.616616616616625,
            "count_ratio": 2.0,
            "matched": 35,
            "local_only": 34,
            "gemini_only": 0,
            "splits": 4,
            "merges": 0,
            "mean_iou": 0.5294639963484731,
            "gem_rallies_in_merges": 0
          },
          "W1+W2": {
            "n": 75,
            "play": 594.494494494495,
            "mean_dur": 7.926593259926601,
            "max_dur": 16.616616616616625,
            "count_ratio": 1.9230769230769231,
            "matched": 35,
            "local_only": 31,
            "gemini_only": 0,
            "splits": 4,
            "merges": 0,
            "mean_iou": 0.5294639963484731,
            "gem_rallies_in_merges": 0
          }
        },
        "status": "ok"
      },
      {
        "name": "GX010129",
        "vid": "99a3e73afde3",
        "csv": "C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\.claude\\worktrees\\w2-safeguard\\output\\wasb\\GX010129__99a3e73afde3_wasb.csv",
        "n_gem": 27,
        "play_gem": 195.6,
        "mean_gem": 7.2444444444444445,
        "variants": {
          "baseline": {
            "n": 2,
            "play": 534.4177510844178,
            "mean_dur": 267.2088755422089,
            "max_dur": 522.6893560226894,
            "count_ratio": 0.07407407407407407,
            "matched": 0,
            "local_only": 1,
            "gemini_only": 0,
            "splits": 0,
            "merges": 1,
            "mean_iou": 0.0,
```

```
            "gem_rallies_in_merges": 27
          },
          "W1": {
            "n": 50,
            "play": 463.7137137137137,
            "mean_dur": 9.274274274274273,
            "max_dur": 27.84451117784451,
            "count_ratio": 1.8518518518518519,
            "matched": 23,
            "local_only": 18,
            "gemini_only": 0,
            "splits": 4,
            "merges": 0,
            "mean_iou": 0.4819655005521238,
            "gem_rallies_in_merges": 0
          },
          "W1+W2": {
            "n": 47,
            "play": 447.0303636970303,
            "mean_dur": 9.511284333979368,
            "max_dur": 27.84451117784451,
            "count_ratio": 1.7407407407407407,
            "matched": 22,
            "local_only": 16,
            "gemini_only": 1,
            "splits": 4,
            "merges": 0,
            "mean_iou": 0.48514996335815863,
            "gem_rallies_in_merges": 0
          }
        },
        "status": "ok"
      },
      {
        "name": "GX020125",
        "vid": "407fa7e7e9f1",
        "csv": "C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\.claude\\worktrees\\local-noai\\output\\wasb\\GX020125__407fa7e7e9f1_wasb.csv",
        "n_gem": 11,
        "play_gem": 71.0,
        "mean_gem": 6.454545454545454,
        "variants": {
          "baseline": {
            "n": 6,
            "play": 280.6806806806807,
            "mean_dur": 46.78011344678012,
            "max_dur": 82.66599933266599,
            "count_ratio": 0.5454545454545454,
            "matched": 1,
            "local_only": 2,
            "gemini_only": 0,
            "splits": 0,
            "merges": 3,
            "mean_iou": 0.40270862800565765,
            "gem_rallies_in_merges": 10
          },
          "W1": {
            "n": 18,
            "play": 254.03737070403744,
            "mean_dur": 14.113187261335414,
            "max_dur": 40.05672339005673,
            "count_ratio": 1.6363636363636365,
            "matched": 9,
            "local_only": 8,
            "gemini_only": 0,
```

```
            "splits": 0,
            "merges": 1,
            "mean_iou": 0.4329964346464835,
            "gem_rallies_in_merges": 2
          },
          "W1+W2": {
            "n": 15,
            "play": 246.5131798465132,
            "mean_dur": 16.434211989767547,
            "max_dur": 40.05672339005673,
            "count_ratio": 1.3636363636363635,
            "matched": 9,
            "local_only": 5,
            "gemini_only": 0,
            "splits": 0,
            "merges": 1,
            "mean_iou": 0.4329964346464835,
            "gem_rallies_in_merges": 2
          }
        },
        "status": "ok"
      },
      {
        "name": "mahadevpura-0",
        "vid": "6460c2fc7647",
        "csv": "C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\.claude\\worktrees\\w2-safeguard\\output\\wasb\\mahadevpura-0__6460c2fc7647_wasb.csv",
        "n_gem": 40,
        "play_gem": 262.0,
        "mean_gem": 6.55,
        "variants": {
          "baseline": {
            "n": 19,
            "play": 711.5615615615616,
            "mean_dur": 37.45060850324008,
            "max_dur": 134.13413413413414,
            "count_ratio": 0.475,
            "matched": 6,
            "local_only": 4,
            "gemini_only": 0,
            "splits": 0,
            "merges": 9,
            "mean_iou": 0.3446682872977475,
            "gem_rallies_in_merges": 34
          },
          "W1": {
            "n": 62,
            "play": 646.1127794461132,
            "mean_dur": 10.421173862034085,
            "max_dur": 47.66433099766434,
            "count_ratio": 1.55,
            "matched": 34,
            "local_only": 22,
            "gemini_only": 0,
            "splits": 2,
            "merges": 2,
            "mean_iou": 0.4928109196809172,
            "gem_rallies_in_merges": 4
          },
          "W1+W2": {
            "n": 43,
            "play": 549.5161828495164,
            "mean_dur": 12.779446112779452,
            "max_dur": 47.66433099766434,
            "count_ratio": 1.075,
```

```
            "matched": 30,
            "local_only": 9,
            "gemini_only": 5,
            "splits": 1,
            "merges": 2,
            "mean_iou": 0.4837616679798798,
            "gem_rallies_in_merges": 4
          }
        },
        "status": "ok"
      },
      {
        "name": "mahadevpura-1",
        "vid": "38bef75d78cb",
        "csv": "C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\.claude\\worktrees\\local-noai\\output\\wasb\\mahadevpura-1__38bef75d78cb_wasb.csv",
        "n_gem": 9,
        "play_gem": 51.2,
        "mean_gem": 5.688888888888889,
        "variants": {
          "baseline": {
            "n": 1,
            "play": 174.2742742742743,
            "mean_dur": 174.2742742742743,
            "max_dur": 174.2742742742743,
            "count_ratio": 0.1111111111111111,
            "matched": 0,
            "local_only": 0,
            "gemini_only": 0,
            "splits": 0,
            "merges": 1,
            "mean_iou": 0.0,
            "gem_rallies_in_merges": 9
          },
          "W1": {
            "n": 2,
            "play": 173.79045712379047,
            "mean_dur": 86.89522856189524,
            "max_dur": 101.88521855188523,
            "count_ratio": 0.2222222222222222,
            "matched": 0,
            "local_only": 0,
            "gemini_only": 0,
            "splits": 0,
            "merges": 2,
            "mean_iou": 0.0,
            "gem_rallies_in_merges": 9
          },
          "W1+W2": {
            "n": 2,
            "play": 173.79045712379047,
            "mean_dur": 86.89522856189524,
            "max_dur": 101.88521855188523,
            "count_ratio": 0.2222222222222222,
            "matched": 0,
            "local_only": 0,
            "gemini_only": 0,
            "splits": 0,
            "merges": 2,
            "mean_iou": 0.0,
            "gem_rallies_in_merges": 9
          }
        },
        "status": "ok"
      },
```

```
      {
        "name": "mahadevpura-2",
        "vid": "038e0e0915a2",
        "csv": "C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\.claude\\worktrees\\local-noai\\output\\wasb\\mahadevpura-2__038e0e0915a2_wasb.csv",
        "n_gem": 39,
        "play_gem": 266.0,
        "mean_gem": 6.82051282051282,
        "variants": {
          "baseline": {
            "n": 5,
            "play": 605.9059059059059,
            "mean_dur": 121.18118118118119,
            "max_dur": 297.03036369703034,
            "count_ratio": 0.1282051282051282,
            "matched": 2,
            "local_only": 0,
            "gemini_only": 0,
            "splits": 0,
            "merges": 3,
            "mean_iou": 0.2945165777074605,
            "gem_rallies_in_merges": 37
          },
          "W1": {
            "n": 40,
            "play": 551.0844177510842,
            "mean_dur": 13.777110443777104,
            "max_dur": 43.3266599933267,
            "count_ratio": 1.0256410256410255,
            "matched": 12,
            "local_only": 4,
            "gemini_only": 0,
            "splits": 4,
            "merges": 5,
            "mean_iou": 0.4204774017180335,
            "gem_rallies_in_merges": 11
          },
          "W1+W2": {
            "n": 40,
            "play": 551.0844177510842,
            "mean_dur": 13.777110443777104,
            "max_dur": 43.3266599933267,
            "count_ratio": 1.0256410256410255,
            "matched": 12,
            "local_only": 4,
            "gemini_only": 0,
            "splits": 4,
            "merges": 5,
            "mean_iou": 0.4204774017180335,
            "gem_rallies_in_merges": 11
          }
        },
        "status": "ok"
      }
    ]
```


15. assistant (2026-06-16T15:15:03.173Z)

I have the data. Now let me verify each part of the claim against the mahadevpura-1 entry (lines 214-266).

Let me extract the relevant numbers for mahadevpura-1:

Baseline: n=1, mean_dur=174.27s, matched=0, mean_iou=0.0, gem_rallies_in_merges=9
W1: n=2, mean_dur=86.895s, max_dur=101.885s, matched=0, mean_iou=0.0,

gem_rallies_in_merges=9
W1+W2: n=2, mean_dur=86.895s, max_dur=101.885s, matched=0, mean_iou=0.0,
gem_rallies_in_merges=9 (identical to W1)

Now let me check each claim element against the data:

Claim 1 ? "baseline n=1 (mean 174.3s)": Data: n=1, mean_dur=174.27. ? Correct.

Claim 2 ? "W1 n=2 (mean 86.9s, max 101.9s -- far above the 12s cap)": Data: W1 n=2,
mean_dur=86.895, max_dur=101.885. ? Correct. Max is far above 12s cap.

Claim 3 ? "W1+W2 unchanged n=2": Data: W1+W2 is byte-identical to W1 (n=2, same play, mean,
max, matched, iou). ? Correct.

Claim 4 ? "matched=0, mean_iou=0.0 across all variants": Data: baseline matched=0/iou=0.0,
W1 matched=0/iou=0.0, W1+W2 matched=0/iou=0.0. ? Correct.

Claim 5 ? "9 gem rallies trapped in merges": Data: n_gem=9; gem_rallies_in_merges=9 in all
three variants. ? Correct.

Now the load-bearing interpretive claims. Let me scrutinize the strongest of them, since the
verdict hinges on whether the cap-never-fired interpretation is actually proven by these
numbers.

**Claim 6 ? "W1's per-video mean window of 86.9s vs the 12s cap proves the cap never fired -- no
internal inactivity gap exists to split on."**

This is the interpretive crux. Let me reason about it carefully and look for a refutation.

The W1 logic (per the WINDOWING KNOBS description): W1 = max_window_duration cap (12s) split at
the largest internal inactivity gap. The baseline produced 1 window of 174.27s. W1 produced 2
windows totaling ~173.79s (mean 86.9s, max 101.9s). So W1 *did* split the blob once (1?2
windows), but every resulting window is still ~87s on average and 101.9s max ? vastly above the
12s cap.

Let me check the arithmetic on the W1 split to make sure the "split-at-gap-then-still-huge"
story is internally consistent.

16. user (2026-06-16T15:15:05.778Z)

W1 total play: 173.79045712379047
mean*n: 173.79045712379047
largest window (max_dur): 101.88521855188523
implied other window: 71.90523857190524
both windows >> 12s cap? True
baseline single window: 174.2742742742743
W1 split count: 1 -> 2

17. assistant (2026-06-16T15:15:13.267Z)

The arithmetic confirms: W1 split the baseline's single 174s blob into exactly 2 windows of
~101.9s and ~71.9s ? both still enormously above the 12s cap.

Now let me adversarially probe the *interpretation*. The claim makes two specific causal
assertions I should test against the data:

**(A) "W1's split-at-largest-inactivity-gap logic cannot cut it because the clip is continuously
active."**
(B) "the cap never fired."

Here is the tension I need to scrutinize. The data shows W1 *did* produce 2 windows, not 1. So
something split the blob once. Under the stated W1 algorithm, the only splitting mechanism is
"split at the largest internal inactivity gap." If the clip were *truly* continuously active
(zero inactivity gaps), W1 would have left it as 1 window (like baseline). The fact that it
became 2 windows means there *was* at least one inactivity gap that W1 split on ? so the clip is

NOT strictly "continuously active."

Let me also examine the precise semantics of the W1 algorithm to be sure whether the cap "firing" and the gap-split are the same operation or distinct.

18. user (2026-06-16T15:15:14.186Z)

Found 8 files
backend\eval\rally_seg_eval.py
backend\pipeline\segmenters\fusion_hybrid.py
tests\test_windowing.py
tests\test_tracknet_runner.py
backend\pipeline\detectors\tracknet_runner.py
backend\pipeline\segmenters\trajectory_hybrid.py
backend\eval\windowing.py
backend\config\models.py

19. user (2026-06-16T15:15:17.223Z)

```
1       """Single-sourced candidate-windowing presets ? the eval?serve foundation fix.
2
3       **The bug this closes.** The offline eval substrate (``distill_local.build_candidates``
?
4       ``fusion_golden`` ? the golden feature JSONs the ablation/compare path re-scores) was
built
5       on a *recall-oriented PROPOSAL* windowing (``vt=0.005, merge_gap=1.0, min_window=0.5``):
6       deliberately permissive so a learned scorer/gate can filter the junk. But the SERVED
pipeline
7       (``TrajectoryHybridSegmenter.process_video``) windows COARSELY (``vt=0.02,
merge_gap=2.0,
8       min_window=2.0`` ? the ``config.indexing`` defaults). A cue measured on the proposal set
is
9       measured on candidates **production never serves**, so a "win" need not transfer. This
is
10      exactly how Gate-A scored **+0.074** on the proposal set yet **?0.008** on the served
set
11      (``scratch/GATE_AB_NUMBERS_AND_IMPLICATIONS.md`` vs ``output/gateA_served_verify/``).
12
13      **The fix.** One module owns the two named windowings as :class:`WindowingPreset`s, and
the
14      SERVED preset is **single-sourced from the same Pydantic config defaults production
reads**
15      (:class:`backend.config.models.IndexingConfig`) ? eval and serve can no longer drift.
Helpers
16      build candidates under either preset, and :func:`served_windowing_from_config` lifts the
17      *actual* served windowing out of a live config (overrides included), so an A/B can
target the
18      operating point a given deployment truly serves.
19
20      The companion rule ? *a cue is promotable only if it does NOT regress at the SERVED
21      windowing* ? lives in :mod:`backend.eval.promotion` (which composes ``eval_stats`` +
22      ``per_user_gate``). This module is the **what windowing**; that one is the **promotion
gate**.
23
24      Pure, offline, stdlib + config only. Nothing here changes existing eval behaviour until
a
25      caller opts in (``distill_local`` re-points its module-level presets here, byte-
identically).
26      """
27      from __future__ import annotations
28
29      from dataclasses import dataclass
30      from typing import Any, Dict, List, Mapping, Tuple
31
32      from backend.config.models import IndexingConfig
```

```
33
34      Interval = Tuple[float, float]
35
36      __all__ = [
37          "WindowingPreset",
38          "SERVED",
39          "PROPOSAL",
40          "PRESETS",
41          "served_windowing_from_config",
42          "build_windows",
43          "windows_to_intervals",
44      ]
45
46
47      @dataclass(frozen=True)
48      class WindowingPreset:
49          """A named (velocity_thresh, merge_gap, min_window_duration) windowing operating
point.
50
51          ``velocity_thresh`` ? normalised per-second shuttle velocity above which a frame
counts as
52          "in flight"; ``merge_gap`` ? seconds within which active frames coalesce into one
window;
53          ``min_window_duration`` ? windows shorter than this are dropped. These are exactly
the three
54          knobs ``TrackNetRunner.trajectory_to_action_windows`` takes, so a preset *is* the
windowing.
55          """
56
57          name: str
58          velocity_thresh: float
59          merge_gap: float
60          min_window_duration: float
61          #: 0 = OFF (default). When > 0, windows longer than this are split at their largest
internal
62          #: inactivity gap ? the bounded-windowing over-merge fix (W1,
RALLY_QUALITY_RESEARCH.md §6).
63          max_window_duration: float = 0.0
64
65          def as_tuple(self) -> Tuple[float, float, float]:
66              """``(velocity_thresh, merge_gap, min_window_duration)`` ? the legacy tuple
shape the
67              ``trajectory_to_action_windows`` / ``distill_local`` call sites already speak.
(The
68              bounded-windowing knob ``max_window_duration`` is passed separately by
``build_windows``.)"""
69              return (self.velocity_thresh, self.merge_gap, self.min_window_duration)
70
71          def to_dict(self) -> Dict[str, Any]:
72              return {
73                  "name": self.name,
74                  "velocity_thresh": self.velocity_thresh,
75                  "merge_gap": self.merge_gap,
76                  "min_window_duration": self.min_window_duration,
77                  "max_window_duration": self.max_window_duration,
78              }
79
80
81      def _served_preset_from_defaults() -> WindowingPreset:
82          """The SERVED windowing, lifted from the SAME Pydantic config defaults production
reads.
83
84          ``TrajectoryHybridSegmenter.process_video`` reads exactly these three:
85            - velocity:   ``idx_cfg[<family>].velocity_threshold`` (default 0.02 ?
wasb/fusion/tracknet)
```

```
86            - merge_gap:  ``idx_cfg.dead_time_merge_gap`` (default 2.0)
87            - min window: ``idx_cfg.min_action_window_duration`` (default 2.0)
88
89          Sourcing them from :class:`IndexingConfig`'s defaults (not a hand-typed tuple) is
the whole
90          point: change a served default in ``config/models.py`` and the eval SERVED preset
follows,
91          so the two can never silently diverge again.
92          """
93          idx = IndexingConfig()  # construct with defaults ? the served operating point
94          return WindowingPreset(
95              name="served",
96              velocity_thresh=float(idx.wasb.velocity_threshold),  # == fusion/tracknet
default 0.02
97              merge_gap=float(idx.dead_time_merge_gap),
98              min_window_duration=float(idx.min_action_window_duration),
99              max_window_duration=float(idx.max_window_duration),  # W1 bounded-windowing
(default 0=OFF)
100         )
101
102
103     #: The COARSE windowing production actually serves (config defaults; ~0.02/2.0/2.0). A
cue is
104     #: only promotable if it holds up HERE ? the operating point real users get.
105     SERVED: WindowingPreset = _served_preset_from_defaults()
106
107     #: The recall-oriented PROPOSAL windowing the offline substrate is built on
(deliberately
108     #: permissive: propose many, let the classifier/gate filter). NOT what production serves
? a
109     #: win here must be re-confirmed on SERVED before promotion. Mirrors the historical
110     #: ``distill_local.PROPOSAL`` tuple (0.005/1.0/0.5) so the existing golden JSONs stay
valid.
111     PROPOSAL: WindowingPreset = WindowingPreset(
112         name="proposal", velocity_thresh=0.005, merge_gap=1.0, min_window_duration=0.5
113     )
114
115     #: Name ? preset, so a CLI / config string ("served" | "proposal") selects one.
116     PRESETS: Dict[str, WindowingPreset] = {SERVED.name: SERVED, PROPOSAL.name: PROPOSAL}
117
118
119     def served_windowing_from_config(config: Any, family: str = "wasb") -> WindowingPreset:
120         """The SERVED windowing a *specific* live config serves (overrides honoured).
121
122         Reproduces ``TrajectoryHybridSegmenter.process_video``'s knob reads exactly, against
an
123         arbitrary config (the typed :class:`AppConfig` or a plain dict) and segmenter
``family``
124         (``"wasb"`` / ``"fusion"`` / ``"tracknet"``). Use this when a deployment overrode a
served
125         default in ``config.json`` ? the module-level :data:`SERVED` is the *default*
operating point;
126         this is *this config's* operating point. Falls back to the served defaults for any
absent key.
127         """
128         idx = _as_mapping(config, "indexing")
129         family_cfg = _as_mapping(idx, family)
130         return WindowingPreset(
131             name=f"served:{family}",
132             velocity_thresh=float(family_cfg.get("velocity_threshold",
SERVED.velocity_thresh)),
133             merge_gap=float(idx.get("dead_time_merge_gap", SERVED.merge_gap)),
134             min_window_duration=float(idx.get("min_action_window_duration",
SERVED.min_window_duration)),
135             max_window_duration=float(idx.get("max_window_duration",
```

```
SERVED.max_window_duration)),
136         )
137
138
139     def _as_mapping(obj: Any, key: str) -> Mapping[str, Any]:
140         """``obj[key]`` as a plain mapping, whether ``obj`` is a dict or a typed config
section
141         (``AppConfig.get`` returns nested sections as dicts). Empty mapping if absent/non-
mapping."""
142         if obj is None:
143             return {}
144         if isinstance(obj, Mapping):
145             sub = obj.get(key, {})
146         elif hasattr(obj, "get"):
147             sub = obj.get(key, {})
148         else:
149             sub = getattr(obj, key, {})
150         return sub if isinstance(sub, Mapping) else {}
151
152
153     def build_windows(points: List[Any], fps: float, frame_width: float,
154                       preset: WindowingPreset) -> List[Dict[str, Any]]:
155         """Trajectory ``points`` ? candidate windows under ``preset`` (the one windowing
call site).
156
157         A thin wrapper over ``TrackNetRunner.trajectory_to_action_windows`` that takes a
named preset
158         instead of three loose floats, so eval candidate-building and the served path
reference the
159         SAME knobs by name. Imported lazily to keep this module import-light (no detector
deps)."""
160         from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
161
162         vt, mg, mwd = preset.as_tuple()
163         return TrackNetRunner.trajectory_to_action_windows(
164             points, fps=fps, frame_width=frame_width,
165             velocity_thresh=vt, merge_gap=mg, min_window_duration=mwd,
166             max_window_duration=preset.max_window_duration)
167
168
169     def windows_to_intervals(windows: List[Dict[str, Any]]) -> List[Interval]:
170         """``[{start, end, ...}]`` ? ``[(start, end)]`` ? the interval shape the scorer/gate
speak."""
171         return [(w["start"], w["end"]) for w in windows]
172
```

20. assistant (2026-06-16T15:15:21.266Z)

Now I understand the precise W1 semantics. Let me read the actual `trajectory_to_action_windows` implementation to confirm exactly how `max_window_duration` splitting works ? this is critical to validate the "cap never fired" claim.

21. user (2026-06-16T15:15:23.164Z)

```
142:        # Normalize separators so basename/splitext work when tests (or collector)
146:        stem, ext = os.path.splitext(base)
256:    def trajectory_to_action_windows(
265:        max_window_duration: float = 0.0,
266:        min_split_gap: float = 0.5,
277:        ``max_window_duration`` (0 = OFF, the default ? behaviour unchanged): a guard
against
280:        window longer than this is recursively SPLIT at its largest internal inactivity gap
(the
282:        that gap is at least ``min_split_gap`` seconds. This is the bounded-windowing fix
from
```

```
334:        if max_window_duration > 0:
335:            groups = TrackNetRunner._split_overlong_groups(
336:                groups, int(max_window_duration * fps), int(min_split_gap * fps))
342:    def _split_overlong_groups(groups: List[List[Tuple[int, float]]], max_win_frames: int,
343:                               min_split_frames: int) -> List[List[Tuple[int, float]]]:
344:        """Recursively split any active-frame group longer than ``max_win_frames`` at its
largest
345:        internal gap (? ``min_split_frames``) ? an inactivity-aware cut at the most likely
rally
353:        while work:
363:            if best_i < 0 or best_gap < min_split_frames:
```

22. assistant (2026-06-16T15:15:27.443Z)

This is the key mechanism. Let me read the full `_split_overlong_groups` implementation (lines 342-380ish) to confirm the exact behavior ? specifically what happens when there's no gap ? `min_split_gap`.

23. user (2026-06-16T15:15:28.037Z)

```
256         def trajectory_to_action_windows(
257             points: List[TrajectoryPoint],
258             fps: float,
259             frame_width: float,
260             chunk_start: float = 0.0,
261             velocity_thresh: float = 0.02,
262             merge_gap: float = 2.0,
263             min_window_duration: float = 2.0,
264             chunk_index: Optional[int] = None,
265             max_window_duration: float = 0.0,
266             min_split_gap: float = 0.5,
267         ) -> List[Dict[str, Any]]:
268             """Convert a shuttle trajectory into candidate rally windows.
269
270             A frame is "active" when the shuttle is visible AND its inter-frame
271             displacement (normalised by frame width, per second) exceeds
272             ``velocity_thresh`` ? i.e. the shuttle is in flight. Active frames within
273             ``merge_gap`` seconds of each other are grouped into one window; short
274             windows are dropped. Mirrors HybridYoloSegmenter's windowing so the dict
275             shape feeding the AI-handoff phase is identical.
276
277             ``max_window_duration`` (0 = OFF, the default ? behaviour unchanged): a guard
against
278             the *over-merge* failure where "shuttle moving" is conflated with "rally in
progress"
279             and a single window swallows several rallies + the dead time between them. When
set, any
280             window longer than this is recursively SPLIT at its largest internal inactivity
gap (the
281             longest stretch with no in-flight shuttle ? the most likely rally boundary),
provided
282             that gap is at least ``min_split_gap`` seconds. This is the bounded-windowing
fix from
283             ``docs/RALLY_QUALITY_RESEARCH.md`` §6 (W1); it never merges, only cuts, so
recall cannot
284             drop, and it is config-gated default-OFF until measured on the golden set.
285             """
286             if fps <= 0:
287                 fps = 30.0
288             if frame_width <= 0:
289                 frame_width = 1280.0
290
291             # Compute per-frame normalised velocity from the last *visible* point.
292             active = []  # (frame_idx, velocity)
293             prev: Optional[TrajectoryPoint] = None
```

```
            for p in points:
                if not p.visible:
                    prev = None  # break the trajectory; absent shuttle = not in flight
                    continue
                if prev is not None:
                    dframes = p.frame - prev.frame
                    if dframes > 0:
                        dist = ((p.x - prev.x) ** 2 + (p.y - prev.y) ** 2) ** 0.5
                        velocity = (dist / frame_width) * (fps / dframes)
                        if velocity > velocity_thresh:
                            active.append((p.frame, velocity))
                prev = p

            if not active:
                return []

            max_gap_frames = int(merge_gap * fps)

            def _finalize(group: List[Tuple[int, float]]) -> Dict[str, Any]:
                vels = [v for _, v in group]
                return {
                    "start": group[0][0] / fps + chunk_start,
                    "end": group[-1][0] / fps + chunk_start,
                    "active_frame_count": len(group),
                    "peak_velocity": max(vels),
                    "mean_velocity": sum(vels) / len(vels),
                    "track_id_count": 1,  # single shuttle; kept for window-shape parity
                    "chunk_index": chunk_index,
                }

            groups: List[List[Tuple[int, float]]] = []
            group = [active[0]]
            for af in active[1:]:
                if af[0] - group[-1][0] <= max_gap_frames:
                    group.append(af)
                else:
                    groups.append(group)
                    group = [af]
            groups.append(group)

            if max_window_duration > 0:
                groups = TrackNetRunner._split_overlong_groups(
                    groups, int(max_window_duration * fps), int(min_split_gap * fps))

            windows = [_finalize(g) for g in groups]
            return [w for w in windows if (w["end"] - w["start"]) >= min_window_duration]

        @staticmethod
        def _split_overlong_groups(groups: List[List[Tuple[int, float]]], max_win_frames:
int,
                                   min_split_frames: int) -> List[List[Tuple[int, float]]]:
            """Recursively split any active-frame group longer than ``max_win_frames`` at
its largest
            internal gap (? ``min_split_frames``) ? an inactivity-aware cut at the most
likely rally
            boundary. Splitting only subdivides existing groups (never merges/extends), so
it cannot
            invent or drop coverage; a group with no gap ? the floor is left intact (a
genuinely long
            rally). Iterative worklist to bound stack depth."""
            if max_win_frames <= 0:
                return groups
            out: List[List[Tuple[int, float]]] = []
            work = list(groups)
            while work:
```

```
354                 g = work.pop()
355                 if len(g) < 2 or (g[-1][0] - g[0][0]) <= max_win_frames:
356                     out.append(g)
357                     continue
358                 best_i, best_gap = -1, -1
359                 for i in range(1, len(g)):
360                     gap = g[i][0] - g[i - 1][0]
361                     if gap > best_gap:
362                         best_gap, best_i = gap, i
363                 if best_i < 0 or best_gap < min_split_frames:
364                     out.append(g)  # too long but no real dead-time gap ? keep as one rally
365                     continue
366                 work.append(g[:best_i])
367                 work.append(g[best_i:])
368             out.sort(key=lambda grp: grp[0][0])
369             return out
370
```

24. assistant (2026-06-16T15:15:45.680Z)

Now I have the exact algorithm. This is decisive for the interpretive claims. Let me reason carefully.

The W1 split logic (`_split_overlong_groups`):
1. A group longer than `max_win_frames` (12s) is split at its **largest internal gap**, but only if that gap is ? `min_split_gap` (0.5s).
2. Splitting is **recursive** (worklist-based): each resulting sub-group is re-checked and split again if still > 12s with a qualifying gap.
3. A group with **no gap ? 0.5s** is "left intact (a genuinely long rally)."

Now the critical observation for mahadevpura-1:

- The blob is 174s. W1 split it into 2 windows (101.9s + 71.9s). Both are *still* >> 12s.
- Because the split is **recursive**, if either 101.9s or 71.9s sub-group had *any* further internal gap ? 0.5s, it would have been split again. It was NOT ? they remained at 101.9s and 71.9s.
- So after the **first** split, neither sub-group contains *any* inactivity gap ? 0.5s. The single first split consumed the *only* qualifying gap in the whole 174s blob.

This means the claim's framing is partially imprecise but its core conclusion holds. Let me parse the exact wording.

Claim text: "mahadevpura-1 is an irreducible blob: W1's split-at-largest-inactivity-gap logic cannot cut it because the clip is continuously active, so both the cap and W2 are no-ops there."

There's a tension: the data shows W1 *did* cut it once (1?2 windows). So "cannot cut it" / "continuously active" is **literally false** ? there was exactly one qualifying inactivity gap, and W1 used it. The blob is not strictly "continuously active."

However, the claim's deeper, load-bearing point ? that the **cap is effectively a no-op** ? depends on interpreting "the cap firing" as "reducing windows to ?12s." Let me examine the supporting sentence.

Claim text: "W1's per-video mean window of 86.9s vs the 12s cap proves the cap never fired -- no internal inactivity gap exists to split on."

This says **"the cap never fired"** and **"no internal inactivity gap exists to split on."** Both are **factually contradicted by the data**:
- The cap DID fire at least once (n went 1?2; `_split_overlong_groups` only produces 2 groups by performing a split, which only happens when a window > 12s AND a gap ? 0.5s exists).
- An internal inactivity gap **does** exist (exactly one qualifying gap was found and split on).

Let me verify there is no alternate path producing n=2 without a split firing. Looking at the code: `min_window_duration` filtering (line 339) drops short windows ? it cannot *increase*

count. The merge step (lines 324-332) produced the baseline's grouping. The baseline had n=1, meaning at the served preset the entire 174s coalesced into a single group (all active frames within merge_gap=2.0s of each other). W1 uses the *same* velocity/merge/min knobs plus the cap. So the ONLY thing that turns baseline's 1 group into W1's 2 groups is `_split_overlong_groups`. Therefore the cap demonstrably fired exactly once.

Let me confirm the merge_gap vs min_split_gap relationship to be airtight about "gap exists."

25. user (2026-06-16T15:15:53.297Z)

Baseline groups: 1 -> implies no gap >= merge_gap(2.0s)
W1 groups: 2 -> implies exactly one qualifying gap in [0.5, 2.0)s was split
Recursive non-split of 101.9s & 71.9s -> no further gap >= 0.5s in either half

CLAIM says: cap never fired / no inactivity gap exists -> CONTRADICTED (one split happened)
TRUE statement: cap fired exactly once but is effectively a no-op for the goal
  (2 windows of 72-102s each, still 6-8x over the 12s cap; matched still 0, iou still 0)

26. assistant (2026-06-16T15:16:07.919Z)

I now have a complete, code-grounded picture. Let me also confirm the W2 no-op part of the claim, which is clearly supported (W1+W2 is byte-identical to W1: n=2, same play/mean/max/matched/iou/merges ? W2 dropped nothing).

Let me synthesize the verdict.

What the data supports (correct):
- baseline n=1, mean 174.3s ?
- W1 n=2, mean 86.9s, max 101.9s, far above 12s cap ?
- W1+W2 unchanged (byte-identical to W1) ? W2 is a genuine no-op here ?
- matched=0, mean_iou=0.0 across all three variants ?
- 9 gemini rallies trapped in merges across all variants ?
- The high-level thesis that mahadevpura-1 is effectively irreducible for the goal (getting to ~12s windows that match Gemini's ~5.7s rallies): supported ? best W1 result is still 72-102s windows, 0 matches, 0 IoU.

What the data contradicts (the overstatement):
- "W1's split-at-largest-inactivity-gap logic **cannot cut it**" ? FALSE. It DID cut it: 1?2 windows. The recursive `_split_overlong_groups` only produces 2 groups by performing exactly one split, which requires a qualifying inactivity gap (?0.5s) to exist.
- "because the clip is **continuously active**" ? FALSE / overstated. There is at least one inactivity gap (in [0.5s, 2.0s)); the clip is not continuously active. It's "*too active for the merge_gap to separate but with one sub-second-to-2s lull*."
- "the **cap never fired**" ? FALSE. The cap fired exactly once (that's what turned baseline's 1 window into W1's 2).
- "**no internal inactivity gap exists to split on**" ? FALSE. One gap existed and was split on; the remaining sub-groups merely lack *further* gaps ? 0.5s.
- "both the cap **and W2 are no-ops there**" ? the W2 part is TRUE (byte-identical), but the "cap is a no-op" part is imprecise: the cap *did* fire, it just failed to reduce windows anywhere near the 12s target.

The claim's conclusion (mahadevpura-1 stays an unmatched blob, W2 changes nothing) is right, but the stated *mechanism* and the specific quantitative justification ("mean 86.9s vs 12s cap proves the cap never fired ? no inactivity gap exists") are demonstrably wrong: the cap fired once on a real gap and was simply insufficient. The justifying sentence draws a false inference (mean >> cap does NOT imply the cap never fired; here it fired and still left huge windows).

This is an overstatement/misdescription of the mechanism, so the claim as written does not hold.

27. user (2026-06-16T15:16:20.642Z)

Structured output provided successfully

28. assistant (2026-06-16T15:16:31.019Z)

I verified the claim against the actual data file (`C:\Users\avidu\Projects\badminton-highlight-indexer\output\local_vs_gemini_6.json`, mahadevpura-1 entry, lines 214-265) and cross-checked the W1 algorithm in `C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\w2-safeguard\backend\pipeline\detectors\tracknet_runner.py` (`trajectory_to_action_windows` + `_split_overlong_groups`, lines 256-369) and `backend\eval\windowing.py`.

Verdict: **does not hold** (overstated/misdescribes the mechanism).

All the cited *numbers* check out. The thesis that mahadevpura-1 stays an unmatched blob and that W2 is a no-op is supported. But the claim's stated mechanism is refuted by its own data:

- "W1 cannot cut it / the clip is continuously active" is false ? W1 DID cut it (n: 1?2).
- "the cap never fired / no internal inactivity gap exists" is false ? `_split_overlong_groups` only produces 2 groups by performing exactly one split, and a split only happens when a real internal inactivity gap ? `min_split_gap` (0.5s) exists. Baseline's single 174s window means no gap ? merge_gap (2.0s); W1's split into ~101.9s + ~71.9s means exactly one gap in [0.5s, 2.0s) existed and was used.
- The justifying inference "mean 86.9s >> 12s cap proves the cap never fired" is logically invalid: the cap fired once and was simply insufficient (windows still 6-8x over the cap, recursion stopped only because no *further* ?0.5s gaps remained).

Correct framing: it's a near-irreducible over-merge the cap *under-corrects*, not a clip the cap leaves untouched.